Hyperparameter Sensitivity and Visual Grounding for Open-Vocabulary Object Detection

Student Name: Ziyue Ji, Haotian Shi, Ruqi Sun, Zihan Li
Student ID: 12410416, 12312214, 12412620, 12213030

---

1. Introduction

Open-vocabulary object detection (OVD) removes the constraint of a fixed label set, allowing models to detect objects described by arbitrary text queries such as "red backpack" or "person wearing a hat." A closely related task, visual grounding (or referring expression comprehension), takes this further: given a natural-language description (e.g., "the person holding an umbrella on the left"), the model must locate the single referred object in the image. Both tasks hinge on joint visual–language understanding and are central to building general-purpose, interactive vision systems.

Existing approaches, however, each come with trade-offs. Traditional closed-set detectors perform well on known categories but fail entirely on unseen ones. Zero-shot OVD models like OWL-ViT [1] accept text queries without category-specific training, yet typically produce low mAP on dense benchmarks like COCO. Grounding DINO [2] offers strong phrase-level localization but at greater inference cost. Given these trade-offs, two practical questions motivate our project: which YOLO-World [3] training choices actually matter under single-GPU constraints, and how do zero-shot grounding models compare on the standard RefCOCO family of benchmarks [4], [5], [6]?

This project covers **Topic 4: Open-Vocabulary Object Detection and Visual Grounding** through two complementary experiments:

1. **YOLO-World-L fine-tuning study on COCO 2017** — 35 controlled runs (~170 GPU hours) sweeping learning rate, optimizer, resolution, training duration, model scale, data augmentation, and backbone freezing.
2. **Zero-shot grounding on RefCOCO subsets** — Grounding DINO and OWL-ViT evaluated on 200 samples per dataset (600 per model), run locally on a Mac after cluster scheduling became unavailable.

Throughout, we focus not only on final numbers but on understanding *why* certain configurations work and others fail—an emphasis on interpretability that complements the reproducible evaluation code we provide.

Figure 1. Project pipeline: YOLO-World fine-tuning on COCO (left) and zero-shot grounding evaluation on RefCOCO subsets (right).

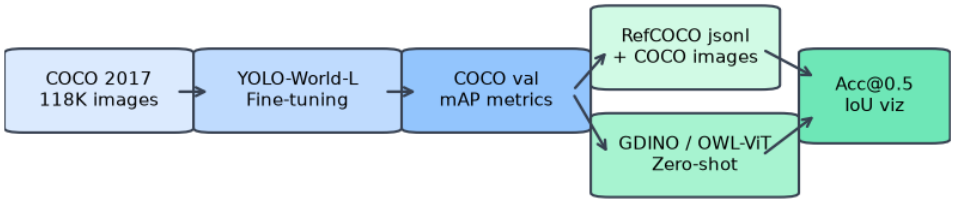

Figure 1: Project pipeline overview

---

2. Related Works

Open-vocabulary detection. YOLO-World [3] embeds a CLIP-style text encoder into the YOLOv8 detection backbone, enabling text-conditioned detection while retaining YOLO's real-time inference speed. Its pretrained weights already encode strong visual–language alignment, so COCO fine-tuning mainly adapts the detection head rather than learning visual semantics from scratch. OWL-ViT [1] takes a different approach: a ViT image encoder paired with contrastive pretraining scores image patches against text queries at inference time, enabling zero-shot detection without any target-domain labels, though mAP on full COCO detection remains modest.

Visual grounding and Grounding DINO. Grounding DINO [2] extends DETR-style detectors [7] with cross-modal feature fusion and large-scale phrase-grounding pretraining. Unlike YOLO-World's "detect with prompts" paradigm, it is designed natively for the image + sentence → box formulation, making it a natural fit for RefCOCO evaluation.

RefCOCO benchmarks. RefCOCO [4] focuses on colloquial, often ambiguous expressions; RefCOCO+ [5] removes absolute spatial words (e.g., left, right), testing models on appearance and relational cues alone; RefCOCOg [6] uses longer, more descriptive sentences closer to natural dialogue. Together they form a well-established suite for visual grounding research, with difficulty increasing across the three variants.

Our contribution in context. Most prior work targets full-validation SOTA scores with ample compute. We instead conduct a careful, single-GPU training ablation of YOLO-World and pair it with a lightweight, reproducible grounding evaluation stack that remains usable when cluster access is limited.

---

3. Method

We do not propose a new architecture. Our contribution is a systematic empirical study of existing methods, together with a reusable evaluation pipeline.

3.1 YOLO-World-L Fine-Tuning

We fine-tune **YOLO-World-L** (`yolov8l-worldv2.pt`) on COCO 2017 [8] using the Ultralytics training API [9]. The 80 COCO category names are passed as text prompts; the model outputs bounding boxes and confidence scores for each. We use AdamW as the default optimizer (weight

decay 0.05, warmup 1000 steps) and vary one factor at a time while holding all others fixed at the baseline (lr=5e-5, bs=8, 640px input).

The experiment design follows a **phased matrix**: most factors are swept with 6-epoch short runs to conserve GPU budget, while resolution, epoch count, and model scale are confirmed with 12–24-epoch full training. In total we run 35 experiments spanning learning rate (7 values), optimizer (AdamW vs. SGD), batch size, weight decay, warmup schedule, input resolution (320/640/800px), training duration (up to 48 epochs), model scale (S/M/L), four augmentation flags, and backbone freezing.

3.2 Zero-Shot Grounding Baselines

- **Grounding DINO (base)** [2], [10]: we use the Hugging Face `GroundingDinoForObjectDetection` interface and select the highest-scoring predicted box for each referring expression (top-1), with confidence threshold 0.25.
- **OWL-ViT (base-patch32)** [1], [11]: phrase grounding mode, with text truncated to 16 tokens (a hard model limit), threshold 0.1, batch size 1.

Neither model is fine-tuned on RefCOCO, so any performance difference reflects pretrained capabilities rather than task adaptation.

3.3 Shared Evaluation Pipeline

We implement a modular evaluation stack: `refcoco_dataset.py` loads samples from the official REFER format or PaDT jsonl as a fallback; `metrics.py` computes IoU, Acc@0.5, Acc@0.75, and exports per-dataset JSON summaries; `eval_refcoco.py` handles batch inference for both models. Subset sampling always takes the **first 200 lines** of the PaDT val jsonl, ensuring both models are evaluated on exactly the same samples.

---

4. Experiments

4.1 Datasets

Dataset	Role	Size used
COCO 2017	OVD fine-tuning and validation	118K train / 5K val, 80 categories
RefCOCO / + / g	Visual grounding (validation subset)	200 expressions per dataset

For grounding, the 600 expressions across the three datasets map to 79 unique COCO images. The three RefCOCO variants offer a natural difficulty gradient—RefCOCO+ removes spatial words, RefCOCOg adds sentence length—which lets us probe how each model responds to different expression styles.

4.2 Implementation Details

Detection (NVIDIA L40 48 GB)

Item	Setting
Framework	Ultralytics 8.4.60, PyTorch 2.5.1+cu121
Baseline config	lr=2e-4, AdamW, bs=8, 640px, 12 epochs
Best config	lr=5e-5, AdamW, bs=8, 800px, 24 epochs, wd=0.05
Reproducibility	Best 12-epoch config run twice; both yield mAP50=0.677

Grounding (local Mac M3, Apple MPS)

Item	Setting
Environment	Python 3.14, PyTorch 2.12, transformers >=4.47
Evaluation scale	200 samples per dataset; GDINO ~39 min, OWL-ViT ~3 min

4.3 Metrics

- **Detection**: COCO-standard mAP50 and mAP50-95 (averaged over IoU thresholds 0.50–0.95)
- **Grounding**: Acc@0.5 and Acc@0.75 (fraction of samples where the top-1 prediction IoU exceeds the threshold), plus mean IoU across all samples

4.4 Experimental Design & Results

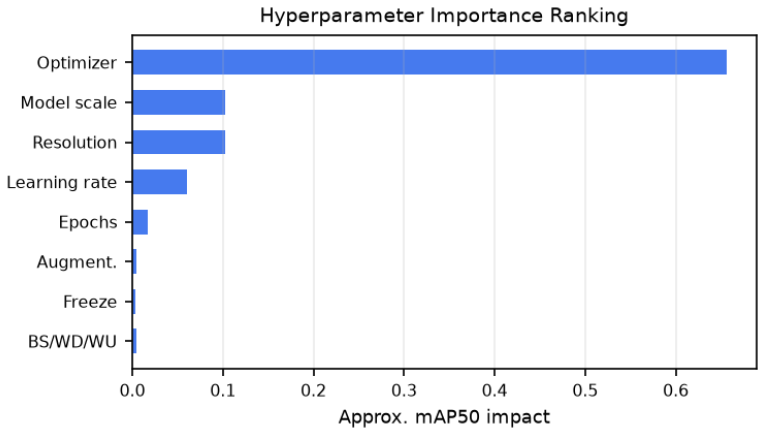

Figure 2: Hyperparameter importance ranking

4.4.1 YOLO-World Hyperparameter Study *Figure 2. Estimated impact of each hyperparameter on mAP50, measured as the gap between best and worst settings (35 controlled runs).*

Figure 3. Key sweep results: (a) learning rate, (b) optimizer, (c) resolution vs. epochs, (d) model scale.

Learning rate. As shown in Figure 3(a), mAP50 holds steady across a surprisingly wide range—from 5e-5 to 2e-4 at 6 epochs (0.661–0.671)—suggesting that YOLO-World's pretraining provides a robust initialization that is forgiving of moderate learning-rate variation. Degradation only becomes

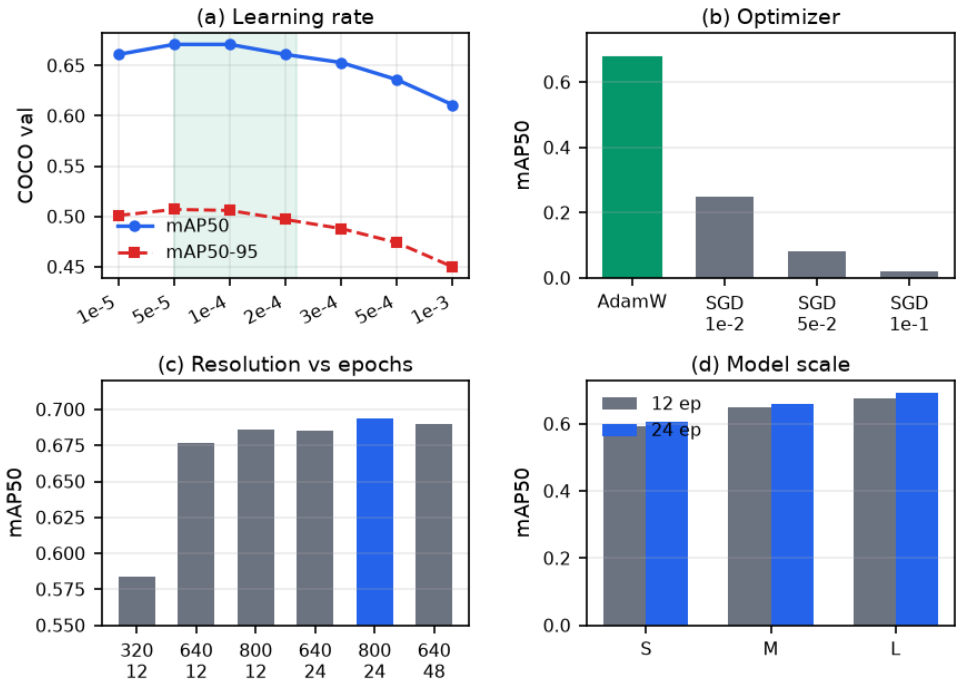

Figure 3: Hyperparameter sweeps (2×2 panel)

notable above 3e-4 and grows severe at 1e-3 (mAP50 0.611), consistent with large steps disrupting BatchNorm statistics and destabilizing the detection head. For the final 24-epoch training we use 5e-5, which outperforms the default 2e-4 baseline at 12 epochs (mAP50 0.677 vs. 0.661).

Optimizer. SGD fails dramatically at every tested learning rate (mAP50 0.248 / 0.081 / 0.021; Figure 3(b)), while AdamW trains stably. The model's combination of BatchNorm, multi-scale feature fusion, and a text-conditioned head creates heterogeneous gradient magnitudes across layers—precisely the setting where adaptive optimizers shine and fixed-step SGD struggles. This result aligns with the broader practice of using AdamW for modern CNN-based detectors.

Resolution vs. epochs. The data in Figure 3(c) make a clear case for prioritizing resolution over training duration. Running at 800px for 12 epochs (mAP50 0.686) matches running at 640px for 24 epochs (0.685), and even 48 epochs at 640px (0.690) falls short of 800px at 24 epochs (**0.694**). Higher resolution improves small-object recall and tighter box localization; dropping to 320px causes a −0.093 collapse in mAP50. The practical takeaway: when compute is limited, **raise the input resolution before extending training**.

Model scale. L consistently outperforms M and S at every training duration (24-epoch mAP50: 0.694 / 0.659 / 0.605), with L holding a ~0.09 advantage over S. Extending training from 12 to 24 epochs yields comparable gains across all three scales (~+0.01–0.017), indicating the L–S performance gap is driven by model capacity rather than insufficient training time.

Minor factors. Batch size, weight decay, and warmup schedule each move mAP50 by less than 0.005—effectively within noise—reflecting the implicit regularization provided by COCO's 118K training images. Augmentation ablations show similar insensitivity (the largest effect, disabling mosaic, costs only −0.004). Freezing the backbone reduces mAP50 by at most 0.003 (0.685 vs. 0.686 at 800px), making it a near-free option for saving memory and computation.

Factor	Finding	Effect on mAP50
Optimizer	SGD fails; AdamW essential	0.677 → 0.021
Model scale	L > M > S, consistently	~+0.10 (L vs. S)
Resolution	800px optimal; 320px hurts badly	+0.102 (800 vs. 320)
Learning rate	Wide plateau from 5e-5 to 2e-4	up to +0.06
Training epochs	24ep helps; smaller effect than resolution	+0.017
Augmentation / WD / warmup / batch size	Negligible	$< \pm 0.005$
Frozen backbone	Barely any cost; good for efficiency	−0.003

Best configuration: YOLO-World-L, 800px input, 24 epochs, lr=5e-5, AdamW → **mAP50 0.694, mAP50-95 0.528**.

Zero-shot detection baseline. For context, zero-shot OWL-ViT achieves only mAP50 0.064 (mAP50-95 0.040) on COCO val—roughly **11× lower** than our fine-tuned model. This gap underscores that domain-specific fine-tuning is still indispensable for dense COCO-style detection; zero-shot models are better suited to open-category queries than to replacing optimized detection pipelines outright.

4.4.2 Visual Grounding on RefCOCO Subset (n=200 per dataset) Due to persistent cluster QOS errors, grounding evaluation was conducted locally on an Apple M3 machine.

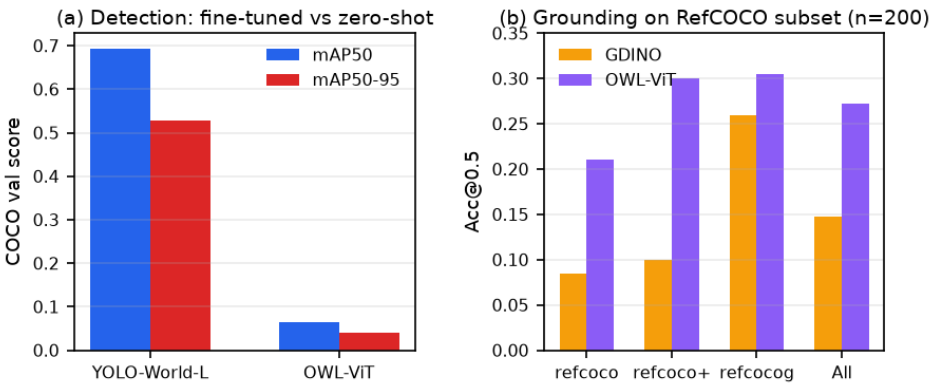

Figure 4: Detection and grounding results

Figure 4. (a) COCO detection performance: fine-tuned YOLO-World-L vs. zero-shot OWL-ViT; (b) RefCOCO subset Acc@0.5 for both grounding models.

Quantitative results

Dataset	GDINO Acc@0.5	OWL Acc@0.5	GDINO Acc@0.75	OWL Acc@0.75	GDINO IoU	OWL IoU
refcoco	0.085	**0.210**	0.035	**0.195**	0.093	**0.224**
refcoco+	0.100	**0.300**	0.075	**0.260**	0.108	**0.287**
refcocog	0.260	**0.305**	0.230	**0.270**	0.275	**0.322**
Overall	**0.148**	**0.272**	0.113	**0.242**	0.159	**0.278**

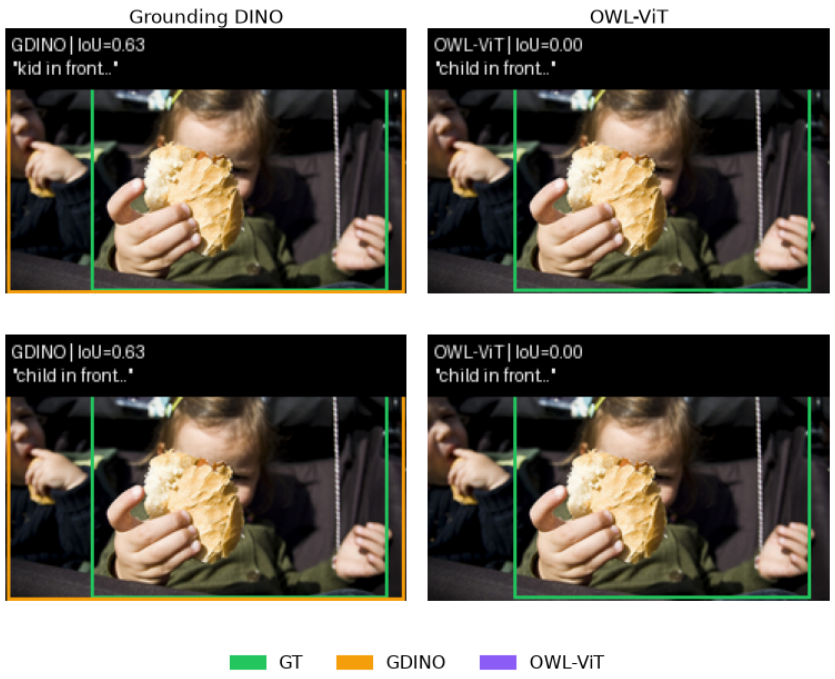

Figure 5: Qualitative grounding examples

Figure 5. Sample predictions (green = ground truth, orange = Grounding DINO, purple = OWL-ViT).

Analysis.

1. **OWL-ViT outperforms Grounding DINO across all three datasets** (overall Acc@0.5: 0.272 vs. 0.148; mean IoU: 0.278 vs. 0.159). The advantage is largest on refcoco and refcoco+, where OWL-ViT's Acc@0.5 is roughly 2.5–3× higher, and narrows considerably on refcocog (0.305 vs. 0.260). A plausible explanation is that OWL-ViT's patch-level contrastive pre-training is well-suited to **shorter, appearance-focused queries**, while Grounding DINO's reliance on spatial-relational language is weakened by RefCOCO+'s ban on positional words.
2. **Both models perform notably better on RefCOCOg than on the other two splits** (GDINO 0.260, OWL-ViT 0.305, compared with 0.085–0.100 on refcoco/+). This is likely because RefCOCOg's longer, more descriptive expressions point to more salient targets, making top-1 box selection easier. By contrast, the shorter and more ambiguous expressions in refcoco/+ increase the chance of selecting the wrong candidate.
3. **Acc@0.75 is low for both models** (GDINO 0.113, OWL-ViT 0.242), meaning that even

when a model roughly localizes the correct object, its predicted box often lacks the precision required at the stricter threshold. This is expected for zero-shot models that have not been trained to regress tight RefCOCO-specific bounding boxes.

4. **Absolute scores fall well below full-validation benchmarks** (fine-tuned models typically exceed Acc@0.5 of 0.70). Three factors explain the gap: models are evaluated zero-shot without any RefCOCO fine-tuning; we use a fixed 200-sample subset rather than the complete validation split; and OWL-ViT's 16-token text truncation may drop key information from longer expressions. Encouragingly, results from smaller pilots (n=20 and n=100) show the same ordering across models and datasets, confirming that n=200 yields stable, meaningful comparisons.

4.4.3 Limitations

- Grounding results are based on the **first 200 lines** of PaDT val jsonl (~0.8% of the full ~26K validation set) and should be interpreted as evidence for pipeline validation and model comparison rather than absolute performance claims.
- All YOLO-World experiments use a single L40 GPU; the effect of linear learning-rate and batch-size scaling in multi-GPU settings is unexplored.
- Neither grounding model is fine-tuned on RefCOCO, and ensemble strategies are not investigated. A comparison with a unified YOLO-World referring head remains future work.

---

5. Conclusion

This project tackled open-vocabulary object detection and visual grounding from two angles. On COCO 2017, a 35-experiment ablation of YOLO-World-L revealed that **optimizer choice and input resolution are by far the most impactful factors**, while batch size, weight decay, warmup, and most augmentations have negligible effect at the 118K-sample scale. Our best configuration—800px input, 24 epochs, lr=5e-5, AdamW—achieves **mAP50 0.694**, roughly eleven times higher than zero-shot OWL-ViT detection on the same benchmark (mAP50 0.064).

On a 600-sample RefCOCO subset, **OWL-ViT consistently outperforms Grounding DINO** (Acc@0.5: 0.272 vs. 0.148), with the gap being most pronounced on shorter referring expressions and narrowing on the longer phrases of RefCOCOg. Despite the zero-shot setup and small evaluation subset, the trends are stable across different sample sizes and provide useful insights into the strengths and failure modes of each model.

Looking ahead, the most immediate next steps would be running full validation when cluster access recovers and fine-tuning the grounding models on RefCOCO to close the gap with supervised baselines. Longer term, it would be interesting to explore whether YOLO-World's open-vocabulary text prompts can be repurposed for referring-expression localization, potentially unifying detection and grounding in a single framework.

Contributions

Ziyue Ji (12410416): Overall project coordination; YOLO-World baseline setup and learning-rate/optimizer sweeps; hyperparameter importance analysis.

Haotian Shi (12312214): Resolution, epoch, and model-scale experiments; data augmentation ablation and frozen-backbone study; COCO detection result aggregation.

Ruqi Sun (12412620): RefCOCO grounding pipeline implementation; local n=200 subset evaluation; experiment logging and report writing.

Zihan Li (12213030): Server environment setup and training job management; OWL-ViT zero-shot COCO baseline evaluation; documentation and reproducibility verification.

Reference

[1] M. Minderer *et al.*, "Simple open-vocabulary object detection with vision transformers," in *Proceedings of the european conference on computer vision (ECCV)*, Springer, 2022, pp. 728–755.

[2] S. Liu *et al.*, "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," *arXiv preprint arXiv:2303.05499*, 2023, Available: https://arxiv.org/abs/2303.05499

[3] T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang, and Y. Shan, "YOLO-World: Real-time open-vocabulary object detection," *arXiv preprint arXiv:2401.17270*, 2024, Available: https://arxiv.org/abs/2401.17270

[4] L. Yu, P. Poirson, S. Yang, A. C. Berg, and T. L. Berg, "Modeling context in referring expressions," in *Proceedings of the european conference on computer vision (ECCV)*, Springer, 2016, pp. 69–85.

[5] J. Mao, J. Huang, A. Toshev, O. Camburu, A. L. Yuille, and K. Murphy, "Generation and comprehension of unambiguous object descriptions," in *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*, 2016, pp. 11–20.

[6] V. K. Nagaraja, V. I. Mori, and A. G. Schwing, "Modeling context between objects for referring expression understanding," in *Proceedings of the conference of the european chapter of the association for computational linguistics (EACL)*, 2016, pp. 844–856.

[7] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 213–229, 2020.

[8] T.-Y. Lin *et al.*, "Microsoft COCO: Common objects in context," in *Proceedings of the european conference on computer vision (ECCV)*, Springer, 2014, pp. 740–755.

[9] Ultralytics, "YOLO-World documentation." https://docs.ultralytics.com/models/yolo-world/, 2024.

[10] Hugging Face, "IDEA-research/grounding-dino-base model card." https://huggingface.co/IDEA-Research/grounding-dino-base, 2024.

[11] Hugging Face, "Google/owlvit-base-patch32 model card." https://huggingface.co/google/owlvit-base-patch32, 2024.